

hotMuscles

A phone app for logging gym workouts and cardio. It shows how many sets each muscle got (today, this week, or a week on average) as a colour heatmap on a body, against targets you pick.



927

exercises

30

muscle regions

7

screens

4

bodies × views

2 · 10 ·
3styles · palettes ·
gradients

12.6k

lines of app code

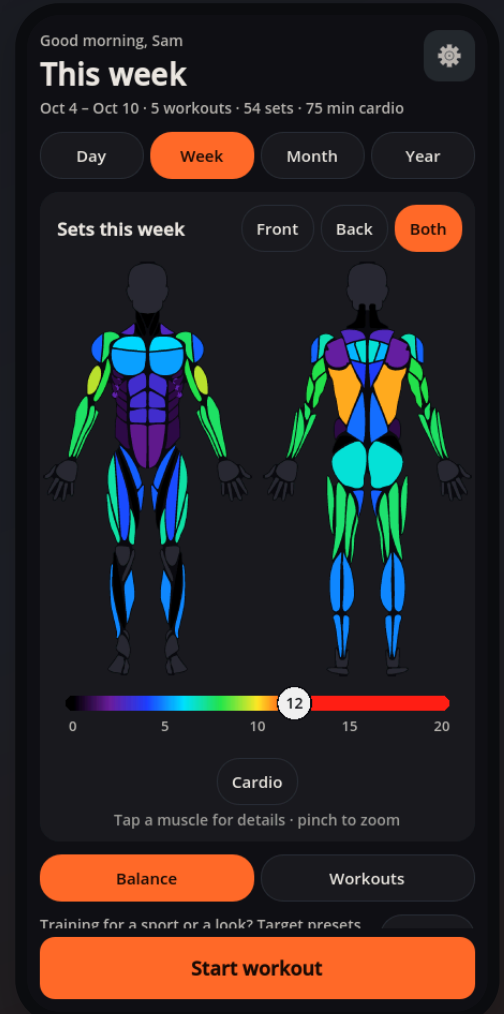
11

target presets

4

home periods (day
→ year)

2 · 3

cardio lights ·
efforts

The home screen: this week's heat on front + back, with the Day | Week | Month | Year chips

What's inside

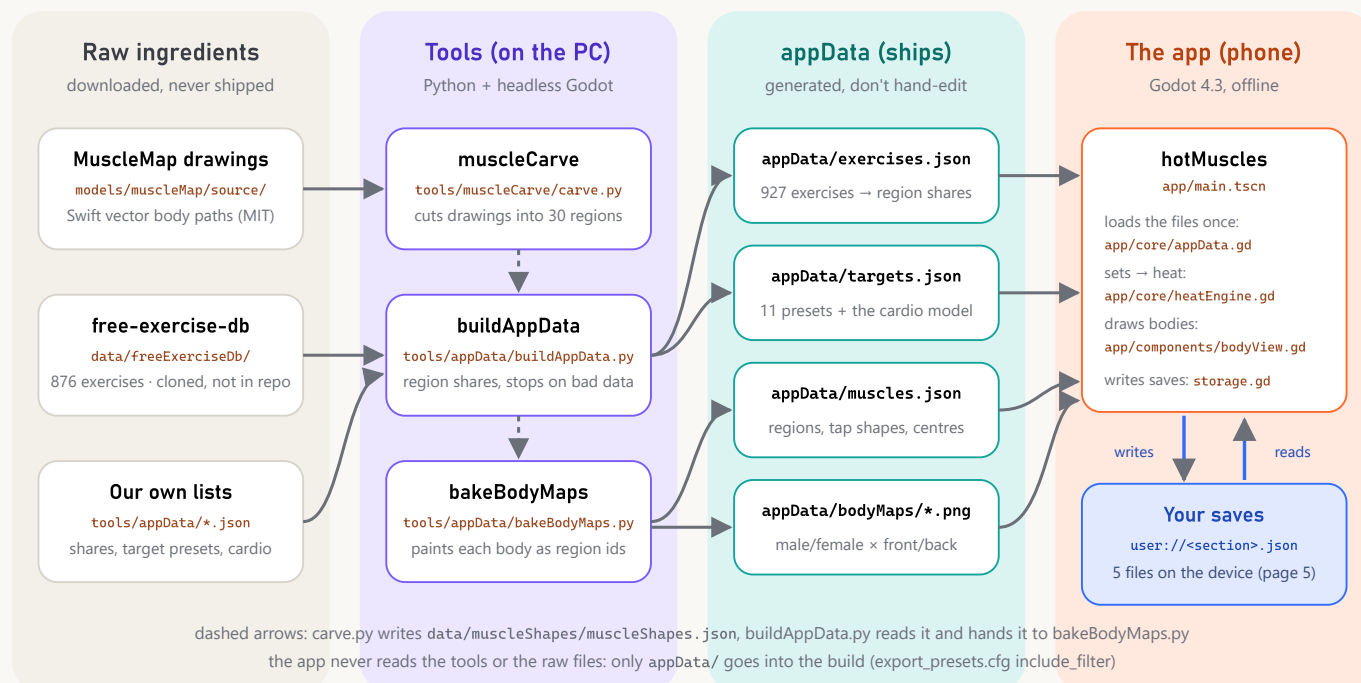
1 Data flow · 2 App structure · 3 Screens and navigation · 4 Saved data · 5 Heat calculation · 6 Targets · 7 Cardio · 8 Home periods and overlays · 9 Body rendering · 10 Looks · 11 Behind the scenes

Built with Godot 4.3 · one codebase → Android apk + Windows exe · fully offline

Reading the file names: every `monospace path` is a real file, counted from the top of the repo (github.com/aphyr-dev/hotMuscles). A name ending in `()` is a function inside the file named next to it.

Data flow: from source files to the app on your phone

Data moves left to right. Body drawings and an exercise list go in on the left. Python tools on the PC turn them into data files, which are packed into the app and read when it starts. The only thing the app writes is your own workout log, saved on the phone.



tools run on the PC only

Re-run `python`
`tools/appData/buildAppData.py`
when a body drawing, an exercise share, a target preset or the cardio model changes. If a share or offset is out of range, an id is unknown, or a muscle can't be reached, it stops and writes nothing.

appData generated, ships

Finished data files, read by `AppData.loadAll()` in `app/core/appData.gd`. The body PNGs ship exactly as made: each has a `.png.import` set to "keep", so Godot never re-compresses them.

saves written by the app

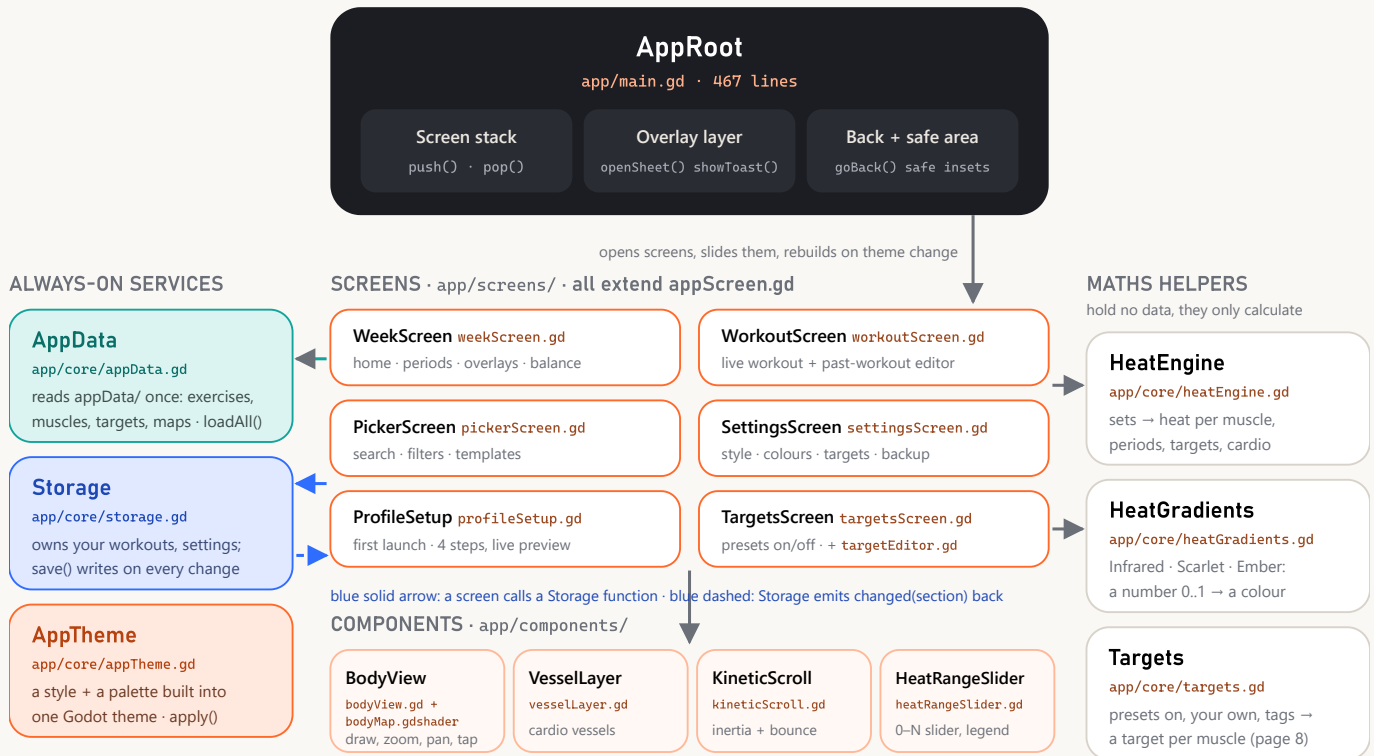
The only files the app writes, all through `app/core/storage.gd`. Small, versioned, crash-safe. Settings → "Export backup" writes them all into one backup file.

Project folders and the files in them

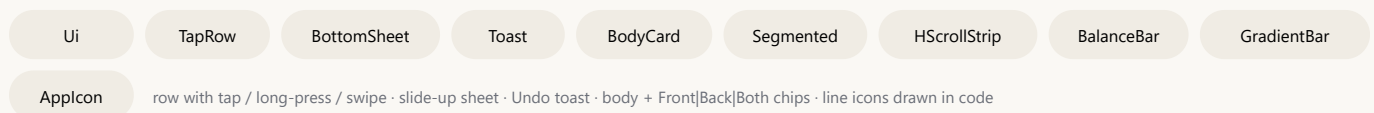
FOLDER	SHIPS?	FILES
app/	in the app	main.gd + main.tscn the root · core/ appData, storage, appTheme, heatEngine, heatGradients, targets · screens/ appScreen (shared base) + the 7 screens (page 3) · components/ bodyView + bodyMap.gdshader, vesselLayer, kineticScroll, heatRangeSlider · ui/ ten small widgets · looks/ styles and palettes (page 12)
appData/	in the app	exercises.json · muscles.json · targets.json · bodyMaps/male_front.png, male_back, female_front, female_back
tools/	PC only	appData/ buildAppData.py, bakeBodyMaps.py + curatedExercises, researchedExercises, targetPresets, cardio (.json) · muscleCarve/ extractShapes.py, carve.py, polygonCut.gd + helpers · build/ buildAll.py, checkBuild.py · icon/ makelcon.py
tests/	PC only	runGodot.py runs them all · test*.gd checks · shot*.gd screenshot drivers (page 13)
data/ · models/	PC only	data/muscleShapes/ sourceShapes.json + muscleShapes.json · data/sources.md where every input came from · models/muscleMap/source/ the MuscleMap Swift files
docs/ · icons/	PC only	docs/appSpec.md the spec · this guide · icons/ launcher PNGs made by makelcon.py · root: project.godot, export_presets.cfg

App structure: main parts and how they connect

AppRoot (`app/main.gd` , the script on `app/main.tscn`) is the top node of the app. It keeps the stack of open screens and the layer for sheets and toasts. Three services stay loaded the whole time (Godot "autoloads", listed in `project.godot`): AppData holds the shipped files, Storage your saves, AppTheme the look. Three helpers, HeatEngine, HeatGradients and Targets, only calculate. Screens are built from shared widgets in `app/ui/` .



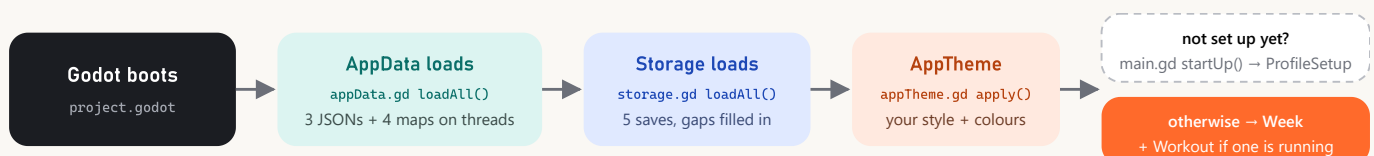
SHARED WIDGET KIT · `app/ui/` · one file each: `TapRow` = `tapRow.gd`, `BottomSheet` = `bottomSheet.gd` ...



Update loop: a screen calls a Storage function such as `Storage.setEntrySets()`. That ends in `save()` in `storage.gd`, which writes the file and emits `changed("workouts")` (a Godot signal: a broadcast message). Each screen connects to it in its `_ready()` and refreshes in `_onStorageChanged()`. Screens never call each other directly.

Look change: `AppTheme.apply()` builds one Godot theme from the picked style + palette (page 12) and emits `themeChanged`. `_onThemeChanged()` in `main.gd` calls `rebuild()` on every open screen (a new style: `_restyle()`).

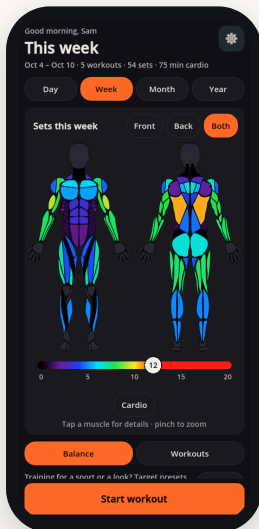
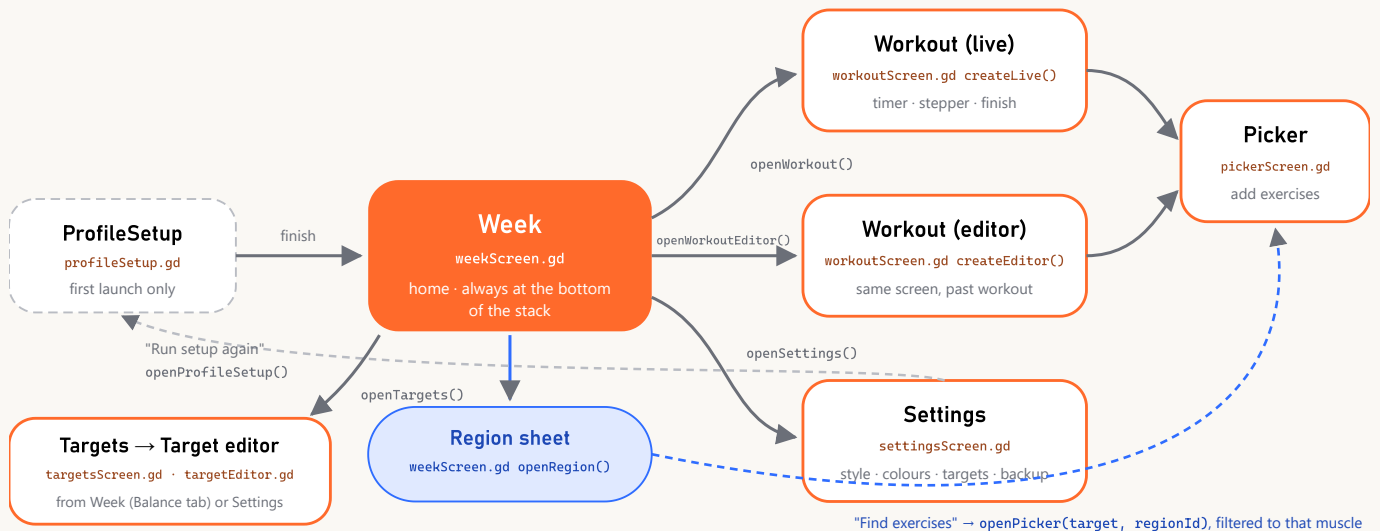
Start-up sequence



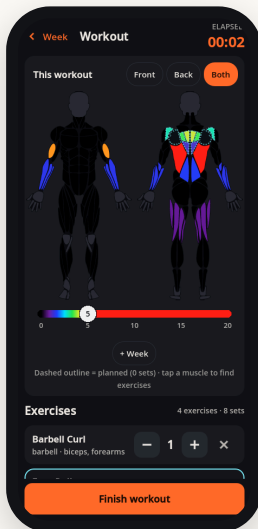
Limit: under 500 ms from the main scene to the first drawn frame. `_onFirstFrame()` in `main.gd` prints the time, and the build check reads it (page 13).

Screens and navigation between them

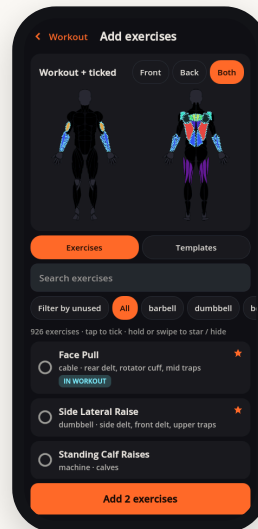
AppRoot keeps the open screens in a stack: `push()` in `app/main.gd` slides a screen in on top, `pop()` slides it away. Week is always at the bottom. Each arrow below is labelled with the `main.gd` function that opens the screen. Bottom sheets (like the region sheet) are not screens: `openSheet()` slides them up from a separate overlay layer.



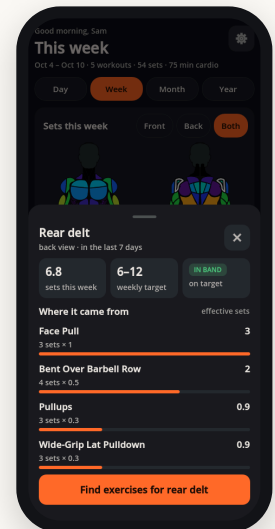
Week



Workout



Picker



Region sheet

Logging one workout: what you tap and which **Storage** function saves it

1 · Start workout

`startWorkout()` stamps a start time; Workout slides in.

2 · Add exercises

The picker opens. Ticked ones show dashed on the body. Nothing saved yet.

3 · Add

`addEntries()` saves them in one write, each made by `newEntry()` (1 set, or 0 by the setting).

4 · - / + sets

`setEntrySets()` saves on every tap; muscles light up. Cardio: `setEntryMinutes()`

5 · Finish

`finishWorkout()` drops rows with no sets and no minutes, stamps the end time.

6 · Back on Week

Week's `_onStorageChanged()` redraws the heat; a toast confirms.

Back button: `main.gd goBack()`, in order

1 Close an open sheet. 2 Ask the screen's `onBack()` (setup steps back, the editor asks about unsaved changes). 3 Otherwise `pop()`. 4 Nothing left: quit (Escape on PC never quits).

Resume and edit: closed mid-workout? `startUp()` in `main.gd` opens Week, then the running workout on top. The editor changes a copy until "Save changes" calls `updateWorkout()`.

Saved data: the five save files and their fields

Each kind of data is saved in its own JSON file in Godot's `user://` folder, all written by `app/core/storage.gd`. `loadAll()` reads all five at start-up and `save()` rewrites one whenever its data changes. Every file is wrapped as `{"version": 1, "data": ...}`, so a future layout change can upgrade old files (`_migrate()`).

Where `user://` is: Android `/data/data/com.aphyr.hotmuscles/files/` (private to the app; uninstalling deletes it) · Windows `%APPDATA%\Godot\app_userdata\hotMuscles\` · backups go in `user://backups/backup_<date-time>.json` plus the clipboard.

profile	profile.json	settings	settings.json	prefs	exercisePrefs.json
defaults: <code>defaultProfile()</code>		defaults: <code>defaultSettings()</code>		per exercise id · defaults: <code>defaultPref()</code>	
name	text	hideUntouched	yes / no	favourite	star in the picker
body	male female	newExerciseSets	0 or 1 (default 1)	hidden	hide from lists
gradient	infrared scarlet ember	defaultView	front back both	grips	remembered checkbox
theme	palette id: ember ... sakura (10)	rangeWeek	N for the week · 12	lastUsed	time, for sorting
style	modern frutiger	rangeWorkout	N for a workout · 5	effort	last cardio effort
setupDone	yes / no	workoutShowWeek	" + Week" chip		
		homePeriod	day ... year		
		activeTargets	presets on		
		customTargets	your own presets		
		overlayTargets	Targets chip		
		overlayCardio	Cardio chip		

workouts.json: a workout is a list of entries, one per exercise (its sets or minutes)

workouts.json	templates.json
{ submitted: [every finished workout], current: the running one, or null }	a list of { id, name, entries } same entry shape, so a template drops straight into a workout
workout id · startedAt · endedAt (0 = still running) entry exerciseld "Face_Pull" sets 3 grips false sets 0 = planned, shown dashed, dropped on finish entry exerciseld "Bent_Over_Barbell_Row" sets 4 grips true grips ticked = straps used, so no forearm heat from this one entry exerciseld "Running_Outdoor" minutes 30 effort "hard" cardio: minutes + effort (easy hard veryHard) instead of sets · no sets and no minutes = planned	backup file all five sections in one JSON, plus the app name, backupVersion, exportedAt <code>storage.gd exportBackup()</code>

Crash-safe writing: `_writeSection()` writes `workouts.json.tmp` first and swaps it in only once it is complete, so a power cut mid-save leaves the old file whole. A file that can't be read is renamed `workouts.json.bad` by `_readSection()` and only that section starts fresh.

Backup import: `isBackup()` checks every section has the right type. `askImport()` in `settingsScreen.gd` shows the workout and template counts and asks you to confirm. Then `exportBackup()` saves your current data first and `importBackupText()` replaces everything. All or nothing.

Saved data: who writes what, and keeping old saves

Screens never touch the files: they call Storage functions, each ending in `save()` (page 3). Below: who calls what, how a file from an older version is brought up to date, and how every delete can be undone.

Which screen writes which file

FILE	WRITTEN BY	STORAGE FUNCTIONS CALLED
<code>profile.json</code>	profileSetup.gd, settingsScreen.gd (name, body, gradient, style, palette)	<code>setProfileValues()</code> · <code>setProfile()</code>
<code>settings.json</code>	settingsScreen.gd, the 0–N slider in <code>app/ui/bodyCard.gd</code> , <code>weekScreen.gd</code> (period, overlays), <code>workoutScreen.gd</code> (+ Week), <code>app/core/targets.gd</code> (presets on, your own)	<code>setSetting()</code>
<code>exercisePrefs.json</code>	pickerScreen.gd (star, hide), <code>workoutScreen.gd</code> (grips, cardio effort)	<code>setFavourite()</code> · <code>setHidden()</code> · <code>setGripsMemory()</code> · <code>setEffortMemory()</code>
<code>workouts.json</code>	<code>weekScreen.gd</code> , <code>workoutScreen.gd</code> , <code>pickerScreen.gd</code>	<code>startWorkout()</code> · <code>addEntries()</code> · <code>setEntrySets()</code> · <code>setEntryMinutes()</code> · <code>setEntryEffort()</code> · <code>finishWorkout()</code> · <code>deleteWorkout()</code> · <code>updateWorkout()</code>
<code>templates.json</code>	week + workout (save), picker (rename, delete)	<code>saveTemplate()</code> · <code>renameTemplate()</code> · <code>deleteTemplate()</code>

Old saves, new fields

`loadAll()` runs each file through `_mergeDefaults()`: stored values stay, keys new in this version come from the defaults. An older `settings.json` gains period "week", no presets and both overlays off, so nothing changes until you pick.

Then `_cleanAll()` fixes types: `_cleanSettings()` keeps the ranges in 1–20, an unknown period becomes "week", the old `colourMode` key goes; `_cleanCustomTargets()` keeps offsets whole, –12..+10; `_cleanActiveTargets()` keeps each id once; `_cleanEntry()` keeps minutes in 0–600, on cardio entries only. `setSetting()` cleans on every change too.

one of your own presets

`settings.json` `customTargets`

made by `Targets.saveCustom()`, same shape as a researched one (page 8)

id "c<time>_<random>"

name up to 28 letters (`nameMaxLength`)

offsets regionId → sets, –12..+10

cardio easyCardio · hardCardio minutes

activeTargets lists the ids that are on, in the order you switched them on. An id that no longer exists (a deleted preset of your own) is skipped by `Targets.activeIds()`, never an error.

Undo instead of "are you sure?"

Each removing function returns what it removed; the toast's Undo button (`showToast()` in `main.gd`) hands it back.

YOU...	SCREEN	REMOVED BY → PUT BACK BY
delete a finished workout	<code>weekScreen.gd</code>	<code>Storage.deleteWorkout()</code> → <code>restoreWorkout()</code>
remove an exercise from a workout	<code>workoutScreen.gd</code>	<code>Storage.removeEntry()</code> → <code>insertEntry()</code> at the same place
discard the running workout	<code>workoutScreen.gd</code>	<code>Storage.discardWorkout()</code> → <code>resumeWorkout()</code>
delete a template	<code>pickerScreen.gd</code>	<code>Storage.deleteTemplate()</code> → <code>restoreTemplate()</code>
hide an exercise	<code>pickerScreen.gd</code>	<code>setHidden(id, true)</code> → <code>setHidden(id, false)</code>
delete a preset of your own	<code>targetsScreen.gd</code> , <code>targetEditor.gd</code>	<code>Targets.deleteCustom()</code> → <code>restoreCustom()</code> , switched on again if it was on

Heat calculation: from exercises to muscle colours

Each exercise in `appData/exercises.json` lists the regions it works and how much of a set counts for each (its **shares**). `HeatEngine.effectiveSets()` in `app/core/heatEngine.gd` multiplies your sets by the shares and adds them up over the period on show (page 10).

`HeatGradients.amountColour()` turns each total into a colour.

exercise record

`appData/exercises.json`

id · name	"Ab_Roller" · Ab Roller
equipment · category	12 kinds · 7 kinds
forearmKind	none grip direct
curated	yes = hand-tuned shares
targets: one share per region (1.0 = a full set)	
upperAbs	1.0
lowerAbs	0.8
serratus	0.4
lats	0.3
obliques	0.3

Where the shares come from

Hand-tuned (96): typed in

`tools/appData/curatedExercises.json`. **Researched (682):** `researchedExercises.json`, each row with a confidence; the 97 low ones show "rough data".

Automatic: only for an exercise no list covers (none today): `buildAppData.py` splits the database's coarse tags (primary 1.0, secondary 0.5) with its `defaultSpread` table.

Stretching, cardio (149): no heat; cardio counts minutes (page 9).

Grips: 201 exercises have `forearmKind` "grip":

`HeatEngine.entryHeat()` skips their forearm share when you tick grips. 35 are "direct" and always count.

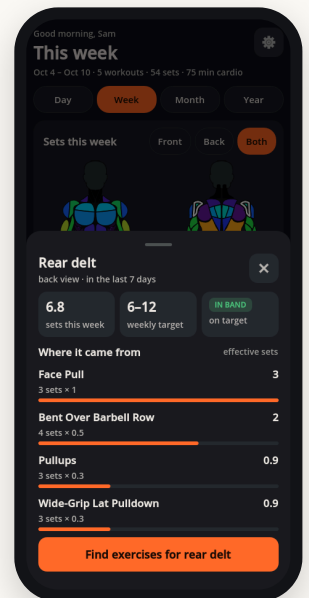
$$\text{heat[muscle]} = \sum \text{sets} \times \text{share}$$

`periodWorkouts()` → `effectiveSets()` → `entryHeat()` per entry

Worked example: rear delt (same numbers as the screenshot →)

Face Pull	3 sets × 1.0	3.0
Bent Over Barbell Row	4 sets × 0.5	2.0
Pullups	3 sets × 0.3	0.9
Wide-Grip Lat Pulldown	3 sets × 0.3	0.9
rear delt this week		6.8
band 6–12 → status "ok"		

The face pull works the rear delt fully (1.0); the row and the lat exercises only assist (0.5, 0.3). Tapping a muscle opens the region sheet (`openRegion()` in `weekScreen.gd`), which lists these rows from `HeatEngine.contributions()`.



Region sheet for the rear delt

Last 7 days, no fading

older: ignored now
`workoutsInWindow()` keeps the 7 × 24 h before now, not the calendar week. Older workouts drop out. Day, Month, Year: page 10.

Heat number to colour

0 6.8 sets, N = 12 N
`amountColour()` samples $t = \text{sets} \div N$ on the `presets` table in `heatGradients.gd`. The slider sets N; above N is the top colour.

Status vs target band

missed under ok over
`targetStatus()` checks each region's band in `muscles.json` (from `targetBands` in `buildAppData.py`; 6–12 for most). Under 0.5 sets is missed. `rankRegions()` sorts the balance list.

30 regions in 7 groups (`regions`, `groups` in `appData/muscles.json`): Back 6 · Shoulders 4 · Chest 3 · Arms 3 · Core 4 · Legs 9 · Neck 1. The picker ranks by `HeatEngine.recommend()`: share (summed over the picked muscles) + 0.5 × focus, plus small bonuses.

Targets: a weekly goal per muscle, from presets

A target preset is a researched goal for a sport or a look. Per muscle it says how many sets a week to add to, or take off, your **baseline** (the home slider's N, 12 by default), plus weekly cardio minutes and key exercises. Switch on any mix: `Targets` in `app/core/targets.gd` turns them into one target per muscle.

target preset

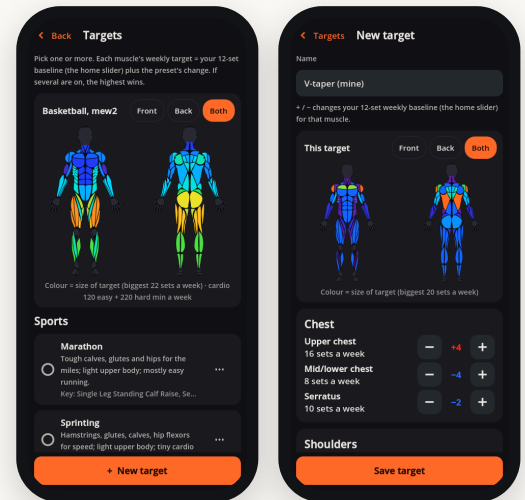
`appData/targets.json`

```
id · name                                "basketball" · Basketball
kind · summary                           sport | physique · one line
offsets                                  all 30 muscles, -12..+10 sets
cardio                                   easyCardio · hardCardio minutes a week
keyExercises                             exercise ids, tagged in the picker
```

Where the presets come from

11 researched (8 sports, 3 physiques) in
`tools/appData/targetPresets.json`, sources in `data/sources.md`.
`buildAppData.py` checks them and writes `appData/targets.json`;
`AppData.presets` holds them.

Your own: `settings.json` `customTargets`, same shape;
`Targets.allPresets()` lists both, `activeTargets` the ids that are on.



Targets screen

Target editor

target[muscle] = highest of max(0, baseline + offset) over the presets on

`Targets.regionTargets()` → `HeatEngine.regionTargets(activePresets(), baseline())` · baseline = `rangeWeek`:
 the slider moves every target

Worked example: Basketball + mew2 on, baseline 12

MUSCLE	BASKETBALL	MEW2	TARGET
Outer quad	+3 → 15	+10 → 22	22
Upper chest	-3 → 9	-8 → 4	9
Biceps	-6 → 6	-9 → 3	6
Easy cardio	120 min	120 min	120 (default 90)
Hard cardio	220 min	75 min	220 (default 30)

Same numbers as the Targets screenshot: "biggest 22 sets a week · cardio 120 easy + 220 hard".

Cardio targets: `Targets.cardioTargets()` → `HeatEngine.cardioTargets()`: per light, the default or the highest preset, whichever is more. The default (`cardioDefaultTarget`) is 90 easy + 30 hard minutes, with hard counting double exactly the WHO minimum of 150 (page 9). With nothing on there are no muscle targets, but the cardio default still applies.

Targets screen

`targetsScreen.gd`: Sports, Physique, Yours. A tap: `toggle()` → `Targets.toggle()` saves `activeTargets`. Hold a row for `openMenu()`: copy a preset into your own, edit or delete yours (`deletePreset()`, undo). The body shows the combined targets (`_refreshPreview()`).

Target editor

`targetEditor.gd`, via `main.gd` `openTargetEditor()`. A stepper per muscle (`changeOffset()`, -12..+10) and easy / hard minutes (`changeCardio()`). `save()` → `Targets.saveCustom()`, a new one switched on; `deleteTarget()` → `Targets.deleteCustom()`, undo `restoreCustom()`.

In the exercise picker

"Below target" chip: `setOrder("target")` weights muscles by `HeatEngine.belowTarget()` (last 7 days + this workout); `weightedFirst()` puts the furthest below first. **Tags:** the picker works out once per list (`presetTags`) which switched-on presets list each exercise as key, and names them on its row.

Cardio: minutes and effort instead of sets

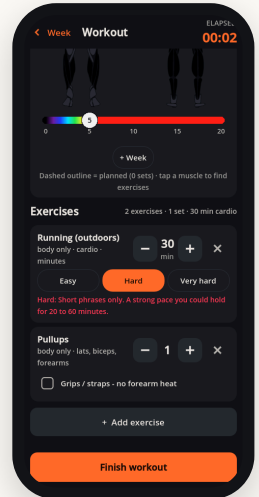
A cardio exercise is logged as minutes and how hard it felt, never as sets, and gives no muscle heat. Its minutes light one of two "cardio lights" instead: blue for easy cardio, red for hard. The model is researched (`tools/appData/cardio.json` , sources in `data/sources.md`) and ships inside `appData/targets.json` next to the presets.

Logging a cardio row: `app/screens/workoutScreen.gd`

The row swaps the sets stepper for minutes (`changeMinutes()` , steps of `minuteStep = 5`) and adds three effort chips with a talk-test hint under them (`setEffort()`). `Storage.newEntry()` starts it at 20 minutes (or 0, by the "new exercises start at" setting) and at the effort you picked last time for that exercise (`setEffortMemory()`), else its default from `cardio.json`. Each change saves through `Storage.setEntryMinutes()` or `setEntryEffort()` .

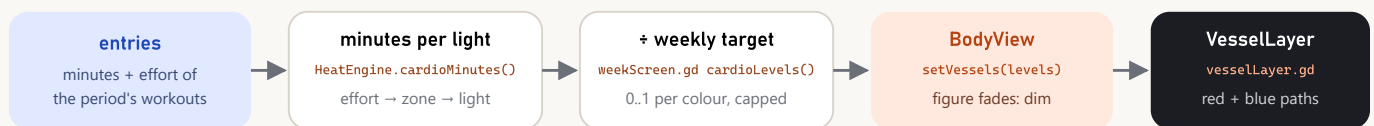
EFFORT	TALK TEST (THE HINT)	ZONE	LIGHTS
Easy	you could chat in full sentences	z1	blue · easy cardio
Hard	short phrases only	z2	red · hard cardio
Very hard	a word or two at most	z3	red · hard cardio

The efforts are `AppData.cardioEfforts / effortById` , the lights `AppData.cardioLights` (each says how much of a minute in each zone lights it). 26 exercises are cardio: 14 from the database, each given a default effort, and 12 added (outdoor run, brisk walk, hike, HIIT circuit...). `HeatEngine.isLogged()` : an entry counts when it has sets *or* minutes; 0 of both is planned and dropped on finish.

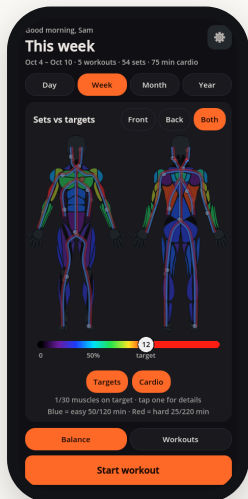


A cardio row (Hard)

From logged minutes to the vessels on the body



Month and year average the minutes per week like the sets (`scaleHeat()` ÷ `periodWeeks()`). A light with no minutes still shows faintly; a lit one carries a travelling pulse.



Cardio overlay (Targets on too)

The vessel layer: `app/components/vesselLayer.gd`

`BodyView.setVessels()` makes a `VesselLayer` on first use, as a child drawn above the figures, and hands it the levels (`setLevels()`). The red and blue paths run through each muscle's left / right anchor (`AppData.bodyView()` anchors), so they follow whichever body and view is shown; the front view gets a beating heart. Under them the shader's `dim` uniform fades the figure toward `dimColour` , the card colour. The style picks the finish (`glassVessels`): Modern draws flat crisp lines, Frutiger glassy glowing tubes (page 12). Colours are a code, not anatomy.

Health minutes = easy + 2 × hard (`HeatEngine.healthMinutes()` , `healthWeights`), measured against the WHO guideline: 150 a week is the minimum (`healthMinimum`), 300 the extra-benefit end (`healthExtra`). The Balance tab's cardio panel (`_fillCardioPanel()` in `weekScreen.gd`) shows both lights as bars against their targets, then this line. Example from the screenshots: 50 easy + 2 × 25 hard = 100 / 150, "Below the WHO minimum".

When the Cardio chip shows: from the first finished workout with any cardio minutes (`Targets.hasCardio()`). Strength sets never count as cardio, and cardio minutes never heat a muscle.

Home periods and the two one-tap overlays

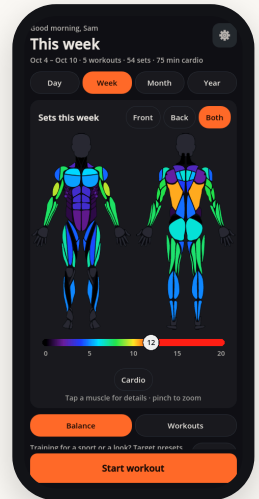
The home page (`app/screens/weekScreen.gd`) can count four time spans, and two chips under the body change what it draws: Targets and Cardio. None of this touches the saved workouts; it only changes which workouts are counted and how the numbers are drawn.

Day | Week | Month | Year: `weekScreen.gd setPeriod()`

PERIOD	WORKOUTS COUNTED	WHAT THE BODY AND THE TABS SHOW
Day	since local midnight (<code>dayStart()</code>)	raw sets; Balance lists only the muscles worked today, as part of their weekly target
Week	the rolling 7 × 24 h (<code>workoutsInWindow()</code>)	raw sets, every muscle on Balance
Month	the rolling 30 days	average sets per week: the total ÷ <code>periodWeeks()</code> , the weeks since your first workout ever (at least 1, at most the whole period)
Year	the rolling 365 days	the same average; the Workouts tab has one row per month, its average week on a small body (<code>_monthBuckets()</code>)

The ids and days are `HeatEngine.periodIds` and `periodDays`. `setPeriod()` saves the pick (`homePeriod`), calls `refresh()` (`periodWorkouts()` → `effectiveSets()` → `scaleHeat()`), also for the cardio minutes) and replays the light-up over exactly 3 s (`startReplay(switchReplaySeconds)`). The 0–N slider stays the weekly scale in all four, so a month's average reads like a normal week.

Start-up replay: with no workout running, opening the app empties the body and replays the period workout by workout, muscle by muscle from the top down (`startReplay()`), at most `replayMaxSeconds = 4.5 s`; a tap on the body skips it.



Week, overlays off

The overlay chips: `BodyCard.addFooterChip()` → `weekScreen.gd _applyOverlays()`

Targets `setTargetsOverlay()`

Shown while a target preset is on (page 8). `displayHeat()` → `HeatEngine.targetHeat()` redraws each muscle as its share of its target on the 0–N scale: at target = N, the hot end; a muscle with no target stays plain. The card title becomes "Sets vs targets" and the hint counts the muscles on target (`HeatEngine.onTarget()`).



The legend: `setScaleMode("target")` sets `HeatRangeSlider.scaleMode`, which numbers the slider 0 / 50% / target / 2× instead of sets. Balance lists only muscles with a target, each band running from the target to 2× it (`_bandOf()`); the region sheet shows "your target".

Cardio `setCardioOverlay()`

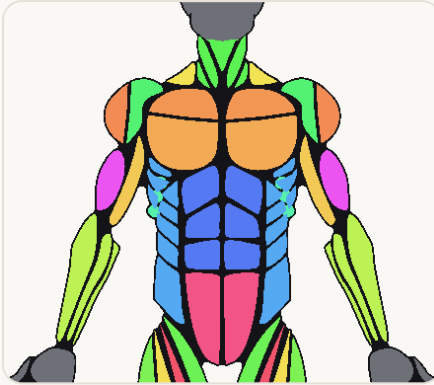
Shown once any cardio was logged (page 9). `cardioLevels()` → `bodyCard.setVessels()` → `BodyView.setVessels()`: red and blue vessels over the body, each as bright as its minutes are of the weekly target. The hint reads "Blue = easy x/y min · Red = hard x/y min" (`cardHintText()`) and the Balance tab gets the cardio panel with the health line.

Both chips

Remembered in settings.json (`overlayTargets`, `overlayCardio`, written once by `_writeOverlay()`), and both can be on together. `_applyOverlays()` does everything a chip changes in one pass: the chips' own state, the card title and hint, `setScaleMode()`, `setHeat(displayHeat(shownHeat))`, `setVessels()` and the Balance tab (`_fillBalance()`). The Balance tab always starts with a targets line: which presets are on, and a button to the Targets screen.

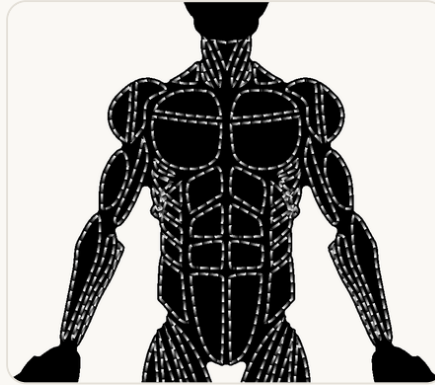
Body rendering: region map texture, palette and shader

Each body is drawn as one textured rectangle by `app/components/bodyView.gd`. The texture is not a picture. It is a region map, `appData/bodyMaps/<body>_<view>.png`, baked by `tools/appData/bakeBodyMaps.py`: each colour channel stores a different number per pixel (like an ID pass in compositing). The shader `app/components/bodyMap.gdshader` reads those numbers and colours the body on every frame. Below: `male_front.png` split into its channels.



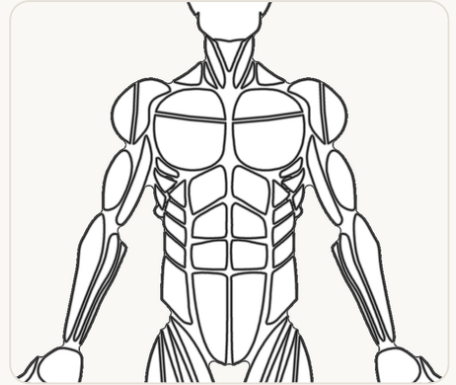
R · which region

the id of the muscle at this pixel: 0 outside the body, 1 seam between regions, 2 head, hands and feet, 3 and up one of the 30 regions (the table is `mapIds` in `muscles.json`). Shown with a random colour per id.



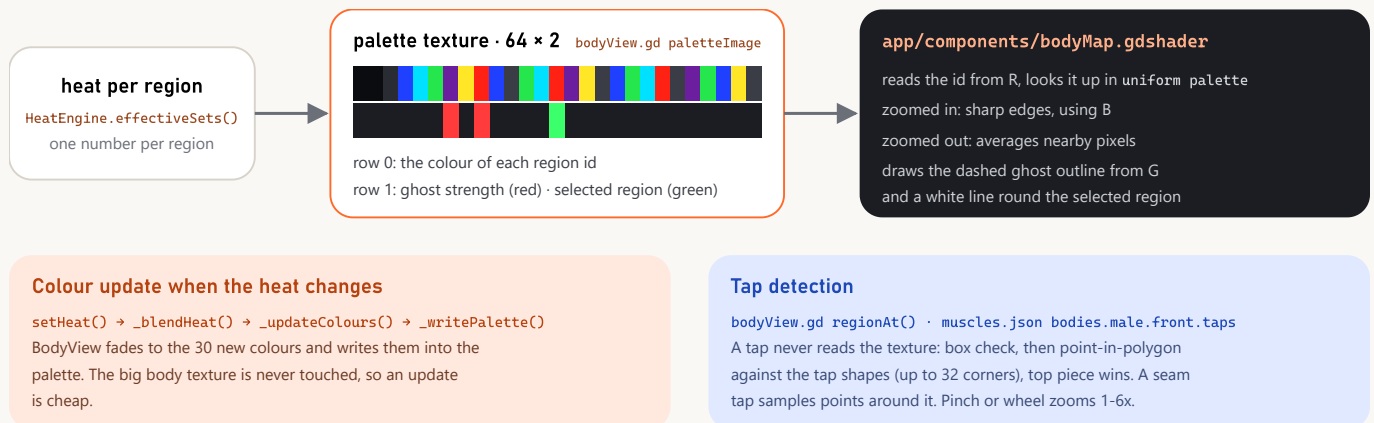
G · position along the outline

A number that runs along each muscle's edge. The shader uses it to draw the dashed "ghost" outline that marks exercises you planned but have not done yet.



B · distance to the edge

How far the pixel is from the nearest edge. The shader uses it to keep muscle edges sharp at any zoom level.



From a logged workout to colours on the Week screen

1 · Save

`Storage.save()` writes
`user://workouts.json`
 and emits
`changed("workouts")`.

2 · Count

`_onStorageChanged()` in
`weekScreen.gd → refresh()`
 →
`HeatEngine.effectiveSets()`:
 one number per region.

3 · Colour

`bodyCard.setHeat() →`
`bodyView.setHeat();`
 each number goes
 through
`amountColour()` into
 the palette.

4 · Draw

`bodyMap.gdshader` looks
 up every pixel's region
 id in the palette, so the
 GPU recolours the body.

Why it works this way: an earlier version drew tens of thousands of outline points per body on every frame. Now the GPU gets one rectangle and two small textures, and the phone builds no geometry at all. If a body map file is missing, `AppData.bodyMap()` logs an error naming the file instead of quietly drawing something else.

Looks: a style and a palette, picked separately

Settings → Look has two separate picks: a **style** (shapes, panels, fonts, the feel) and a **palette** (colours only). Each is one script file in `app/looks/`. AppTheme (`app/core/appTheme.gd`) puts the pair together into one Godot theme. A style never changes what a screen does, only how it looks.

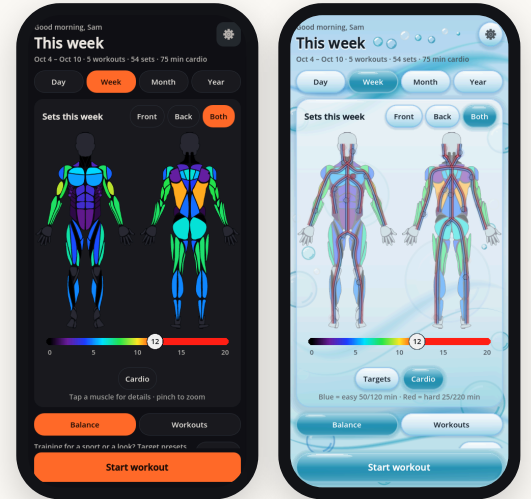
The registry: `app/looks/looks.gd`

`stylePaths` and `palettePaths` list the files in menu order; a file's name is its id (`idOf()`). `loadStyles()` / `loadPalettes()` load them, skipping a broken one with an error in the log. `defaultStyle` "modern" and `defaultPalette` "ember" are worn before you pick. To add a palette: copy a file, change its name and colours, add its path.

Styles: `appStyle.gd` and `styles/`

`app/looks/appStyle.gd` (class AppStyle) is the base and on its own is the Modern look; `styles/modern.gd` only names it. A style overrides only what it changes: shape numbers (radii, paddings, text sizes), `adjustColours()`, `box(role, ...)` for every panel, button and chip, fonts, `decorateFrame()`, `drawCardChrome()`, `glassVessels`.

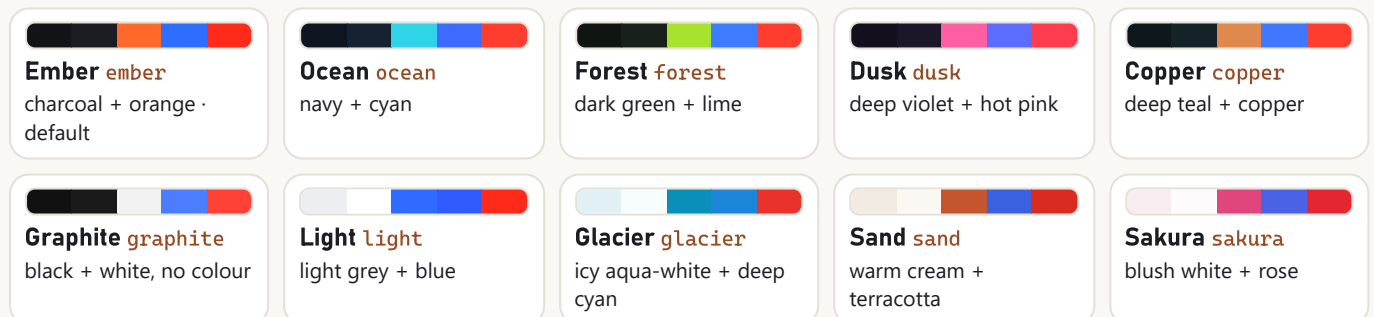
Frutiger (`styles/frutiger.gd`): glossy pills, glass cards, a sky page with bubbles and swooshes, glassy vessels, all drawn in code, with `app/looks/shapeBox.gd` for shapes StyleBoxFlat can't draw.



Modern · Ember

Frutiger · Glacier, cardio on

The ten palettes: `app/looks/palettes/<id>.gd` · swatches: bg, surface, accent, cold, hot



Each palette file holds `paletteName` and 17 `colours`: page, bg, surface, surfaceHi, line, text, textMuted, textFaint, accent, accentText, cold, neutral, hot, ghost, bodyPlain, bodyCosmetic, bodyGap. The body colours feed BodyView's palette texture (page 11); every colour is also in the theme as type "App".

Switching the look: what runs when you tap a new style or palette

1 · Pick

`setStyle()` / `setTheme()` in `settingsScreen.gd` save `Storage.setProfile("style" / "theme")`: the look is part of the profile.

2 · Follow

AppTheme's `_onStorageChanged()` sees the profile differ from what it wears and calls `apply(themeId, styleId)`.

3 · Build

`style.buildTheme(palette)` makes one Godot theme; it goes on the root window, sets the clear colour, emits `themeChanged`.

4 · Redraw

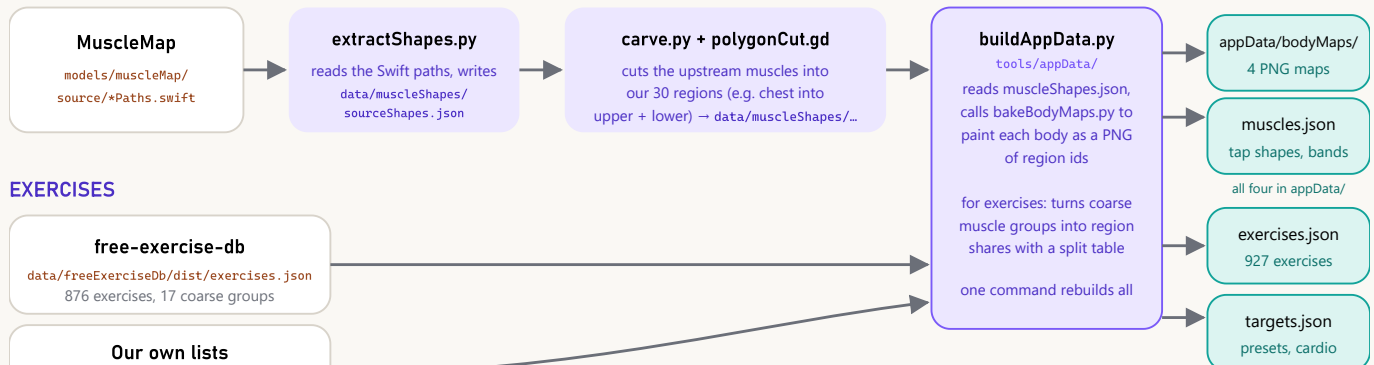
`_onThemeChanged()` in `main.gd`: a new palette → each screen's `rebuild()`; a new style → `_restyle()` builds every screen again, Settings back on top.

Previews without switching: the first-launch setup (`profileSetup.gd`) dresses only itself in the palette you are trying, with `AppTheme.buildTheme()`, which builds a theme without applying it; `finish()` saves the profile and the whole app follows. A saved palette or style that no longer exists falls back to Ember / Modern (`_paletteIdOr()`, `_styleIdOr()`).

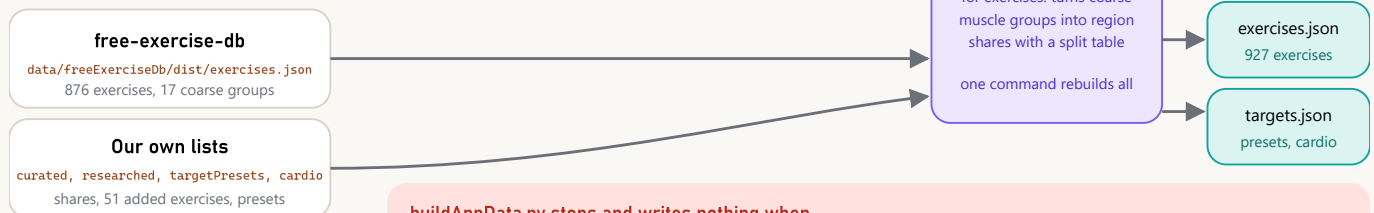
Behind the scenes: building the data and keeping it fast

Two jobs happen out of sight. Scripts in `tools/` build the data files once on the PC, so the phone has almost nothing to compute at start-up. And every build is checked against two time limits, so the app stays quick to open and smooth to use.

BODY SHAPES



EXERCISES



buildAppData.py stops and writes nothing when...

an id is unknown · a share is outside 0-1 · a muscle can't be reached · a preset offset is outside -12..+10

< 500 ms

main scene → first frame, printed by `_onFirstFrame()` in `main.gd`, checked by `tests/testStartup.gd` and `checkBuild.py` (fresh-PC first launch: under 2 s)

< 40 ms

slowest frame in any everyday step (tap, scroll, open a screen): `worstFrameLimitMs` in `testStartup.gd`. Aim 33 ms (30 fps); v0.3 worst was 27-32 ms

How the app meets those limits

- ✓ **Long lists fill in over several frames.** `pickerScreen.gd` builds `firstRows` (6), then `rowsPerFrame` (3) per frame up to `pageSize` (40); all at once was a 90 ms stutter. `weekScreen.gd` (`firstBalanceRows`) and `targetsScreen.gd` do the same.
- ✓ **Update rows, don't rebuild them.** Existing rows are changed or moved in place.
- ✓ **One write for many values.** A write costs about 4 ms, so `Storage.addEntries()` and `setProfileValues()` save many values at once.
- ✓ **Body maps load in the background.** `_startMapDecodes()` in `appData.gd` decodes the PNGs on `ThreadPool` threads.
- ✓ **Idle bodies sleep.** `bodyView.gd` calls `set_process(false)` once nothing on the body is animating.

Automatic checks in `tests/`, all run through `runGodot.py`: `testHeatEngine.gd` (maths), `testStorage.gd` (saves, backups), `testFlows.gd` (every screen with real numbers), `testTouchUi.gd` (synthetic finger), `testKineticScroll.gd`, `testStartup.gd` (timing), `testBuildAppData.py` (bad data refused). `checkBuild.py` also opens each exe three times on a throwaway save folder.

Rebuild and ship

```
python tools/appData/buildAppData.py
python tools/build/buildAll.py
python tools/build/checkBuild.py <exe> <apk>
```

Builds land in `builds/hotMuscles_vNNN` (not in the repo; shared on the Releases page). Every build is a new version and `builds/` keeps every apk ever made. `checkBuild.py` fails any build that holds `data/`, `models/`, `tools/`, `tests/`, `docs/` or `icons/`, or lacks `appData/targets.json`.